TeleportCache: Predictive KV-Cache Prefetching for Low-Latency Multi-Agent Handoffs

Review 2: Implementation Details

Steve Sunny Subhash (24BCE0354)
Ameen S (24BCE0406)
Eldo John Giby (24BCE0387)
Course Faculty: Dr. V. Ezhil Arasi

1. Project Title

TeleportCache: Predictive KV-Cache Prefetching for Low-Latency Multi-Agent Handoffs

2. Project Summary

In a multi-agent LLM system, Agent A passes its context to Agent B. Agent B needs the KV-cache of that context before it can start. Normally the system either recomputes the cache from the tokens (slow) or copies it only after Agent B asks for it (it still waits). In Review 1 we proposed TeleportCache, which treats KV-cache blocks like pages in an Operating System. It predicts the next agent along with a confidence score and copies the needed blocks early, during the short gap between two agents. The amount prefetched depends on the confidence and the available memory.

In Review 2 we implemented this idea as a Python simulator. We compare it with two baselines and an ideal upper bound. The code is on GitHub: https://github.com/Eldo-John-Giby/Predictive-KV-Cache-Prefetching-for-Low-Latency-Multi-Agent-Handoffs

Our design was guided by earlier work on KV-cache reuse, paging and transfer [2], [3], [7]-[12], on multi-agent frameworks [13], [14], and on separating prefill and decode across machines [15]. The OS ideas of working sets and Markov prediction come from [5] and [6].

3. Technology Stack and Programming Languages

Component	Tool	Version	Why we use it
Programming language	Python	3.10 or above	Main code of the simulator, type hints, frozen dataclasses
Numerical computing	NumPy	1.26.x / 2.x	Probabilities, decay scores and budget calculations
Testing	Pytest, Pytest-asyncio	8.4.2	83 unit and integration tests
Plotting	Matplotlib	3.10.x	Latency and hit-rate graphs
Packaging	Setuptools, pyproject.toml	61.0 or above	Installable package and standard project layout
CI	GitHub Actions	v4 / v5	Runs the tests automatically on Python 3.10, 3.11 and 3.12

4. Configuration Details

All hardware-related values are kept in one fixed (frozen) configuration object called SimConfig. The default values are:

Parameter	Default	Meaning
capacity_blocks	16 blocks	Number of KV blocks one agent can keep in memory (acts like limited GPU memory)
bandwidth_bytes_per_s	1.25 x 10^9 B/s	Transfer speed between agents (about 10 Gbps)
transfer_latency_s	0.001 s	Fixed overhead for sending each block
recompute_time_per_token_s	0.001 s	Time to recompute the cache for one token from scratch
prefetch_window_s	0.050 s	Free time between two agents that can be used for prefetching
eviction_policy	lru / fifo	Rule for removing a block when memory is full

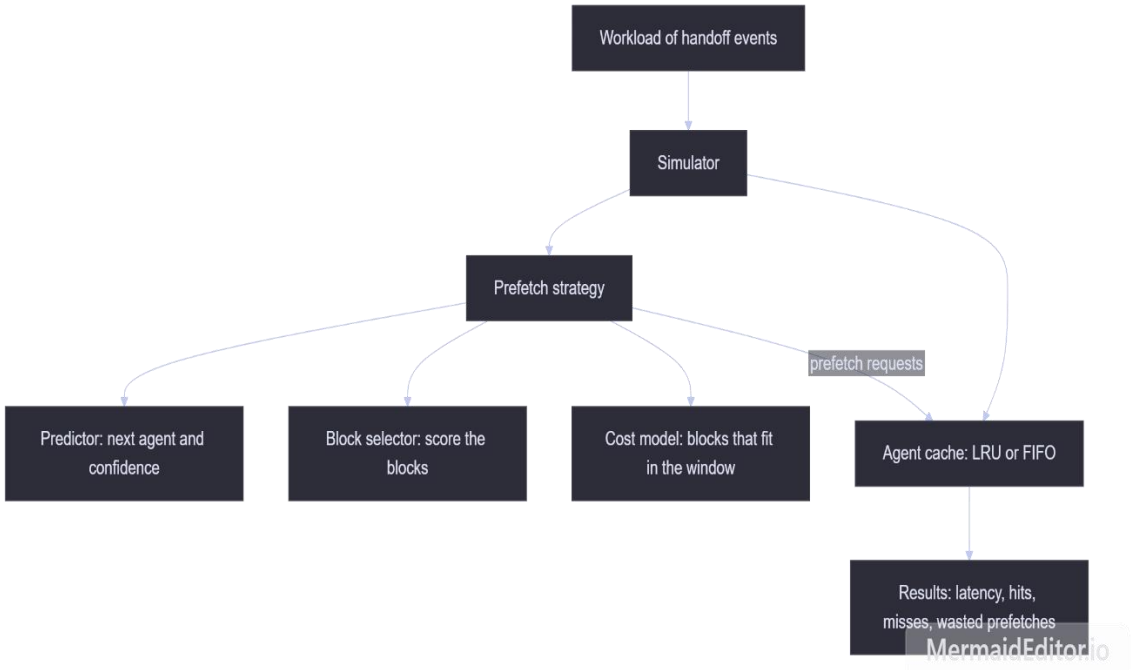
5. Architecture and Modules

The code is divided into small modules, similar to the parts of an OS memory manager.

Module	Role
types	Basic data types: KVBlock (like a page), HandoffEvent, Workload, SimConfig, PrefetchRequest, and the Predictor and Strategy interfaces. A PolicyView object shows only past events, so no strategy can look into the future.
core/cache	Cache of each agent. Uses LRU or FIFO replacement and counts hits, misses and wasted prefetches.
core/cost_model	Calculates the transfer time, recompute time and how many blocks fit in the prefetch window.
core/simulator	Runs the handoff events one by one, does the prefetching, handles misses and produces the results.

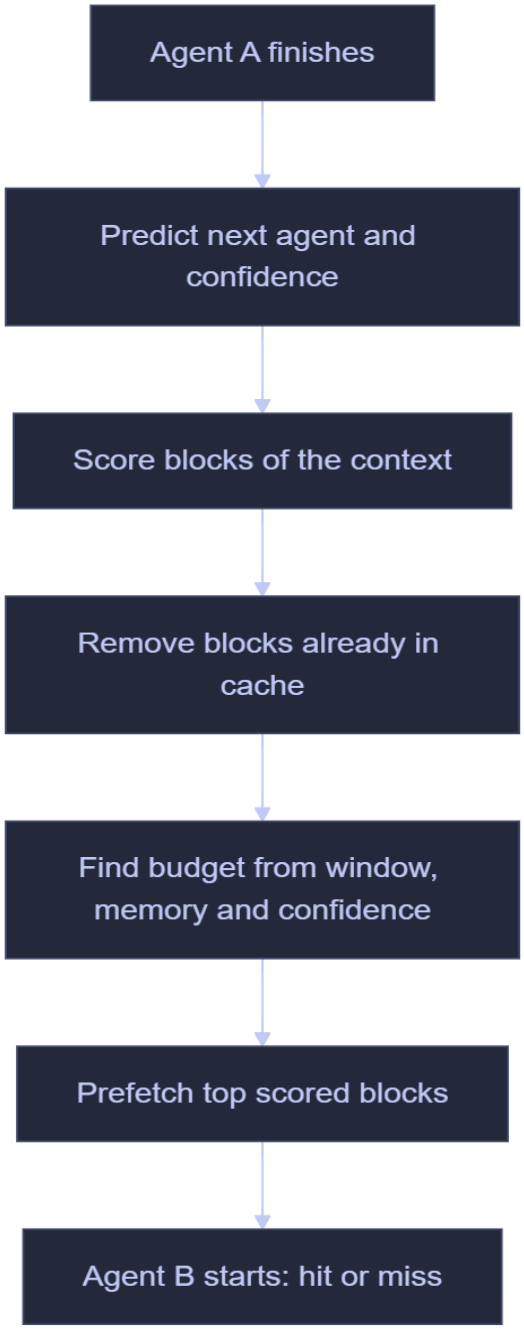
Module	Role
core/baselines	Two comparison methods: Recompute (no cache) and Transfer On Demand.
prediction/predictors	Next-agent predictors: Markov (uses the last k agents), Frequency (counts with add-one smoothing), Random, and NoisyOracle (a predictor with fixed accuracy for testing).
prediction/block_selector	Gives each block a score so that important blocks are prefetched first.
prediction/prefetch_strategy	Main TeleportCache logic: calls the predictor, picks blocks, skips blocks already in memory and sends prefetch requests within the budget.
prediction/oracle	An ideal method that knows the future. It gives the best possible result to compare with.

Figure 1: Module structure of TeleportCache



6. Working of the Implementation

Figure 2: Steps at one handoff



- **Predictor:** the Markov predictor looks at the last k agents and guesses the next one. If it has not seen that sequence before, it falls back to a shorter sequence. The probability it gives is used as the confidence.
- **Block score:** score = usage x 0.5^(age / tau) x boost. A block that is used often and was used recently gets a higher score. Here age is how long ago the block was used and tau controls how fast the score falls.
- **Time cost:** transfer time of a block = overhead + size / bandwidth. Recompute time = number of tokens x time per token.
- **Confidence-aware budget (our additional idea):** the number of blocks that fit in the prefetch window is multiplied by the confidence. High confidence means more blocks are prefetched. Low confidence means few or none, so memory and bandwidth are not wasted.

7. Testing and Results

We wrote 83 unit and integration tests. All of them pass on Python 3.10, 3.11 and 3.12. We then ran a workload of 50 handoff steps and compared the strategies:

Strategy	Total latency	Hits	Misses	Hit rate	Useful prefetches	Wasted prefetches
Recompute (no cache)	102.4000 s	0	200	0.0%	0	0
Transfer on demand	0.3600 s	0	200	0.0%	0	0
Predictive (Frequency)	0.0432 s	176	24	88.0%	176	0
Predictive (Markov-1)	0.0432 s	176	24	88.0%	176	0
Oracle (upper bound)	0.0000 s	200	0	100.0%	200	0

Observations:

- Predictive prefetching reduces latency by about 88% compared with transfer on demand (0.0432 s against 0.3600 s).
- Compared with recomputation (102.4 s) the reduction is more than 99.9%.
- The 24 misses come from the start of the run, when the predictor has not learned the pattern yet. After this warm-up the hit rate is 100% and no prefetch is wasted.
- These results come from our simulator with the configuration in Section 4 and a single workload. They are not measurements on real GPUs. Testing with more workloads and with unpredictable agent patterns is future work.

8. Limitations and Plan for Review 3

- The simulator uses estimated costs, not real GPU timings.
- The workload is regular, so predictors do well. We will test with workloads where the next agent is harder to guess and check how the confidence limits behave.
- For Review 3 we will submit screenshots of the program output, the complete GitHub repository link, and end-to-end multi-agent evaluation traces.

9. References

[1] H. Ye et al., "KVCOMM: Online cross-context KV-cache communication for efficient LLM-based multi-agent systems," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2025.

[2] Z. Pan et al., "KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2025.

[3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. ACM Symposium on Operating Systems Principles (SOSP), 2023.

[4] W. Lee et al., "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.

[5] D. Joseph and D. Grunwald, "Prefetching using Markov predictors," in Proc. International Symposium on Computer Architecture (ISCA), 1997.

[6] P. J. Denning, "The working set model for program behavior," Communications of the ACM, vol. 11, no. 5, pp. 323-333, 1968.

[7] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2024.

[8] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," in Proc. ACM SIGCOMM, 2024.

[9] J. Yao et al., "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in Proc. European Conference on Computer Systems (EuroSys), 2025.

[10] R. Qin et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," in Proc. USENIX Conference on File and Storage Technologies (FAST), 2025.

[11] I. Gim et al., "Prompt Cache: Modular attention reuse for low-latency inference," in Proc. Machine Learning and Systems (MLSys), 2024.

[12] B. Gao et al., "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in Proc. USENIX Annual Technical Conference (ATC), 2024.

[13] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in Proc. Conference on Language Modeling (COLM), 2024.

[14] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proc. International Conference on Learning Representations (ICLR), 2024.

[15] Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.